

Paralelización del Algoritmo K-Nearest Neighbors con MPI y Memoria Compartida OMP: Análisis de Escalabilidad

Ricardo Amiel Acuña Villogas
Departamento de Ciencia de Datos
ricardo.acuna@utec.edu.pe

Camilo Ernesto Soto Cristobal
Departamento de Ciencia de Datos
camilo.soto@utec.edu.pe

Resumen—Este trabajo presenta el diseño, implementación y análisis de rendimiento de una paralelización del algoritmo K-Nearest Neighbors (KNN) sobre el dataset `load_digits` de scikit-learn [1]. El desarrollo se registra en tres etapas incrementales (bucle base, cómputo vectorizado y comunicación por buffers) implementadas con MPI a través de `mpi4py` [2], y se contrasta con una versión de *memoria compartida* (numba con hilos OpenMP) [3] para justificar empíricamente la elección del paradigma. El modelo PRAM se expresa con el formalismo *pardo* para leer el *span* directamente, y el tiempo de comunicación se modela con las colectivas de árbol binomial $T_{comm} = \Theta(\log p(\alpha + \beta m))$. Los experimentos se ejecutan en una máquina de 4 núcleos físicos / 8 hilos (Hyper-Threading), con $p \in \{1, 2, 4, 8\}$ y $n \in \{1797-20000\}$. El speedup MPI satura cerca de $p = 4$ (máximo $S \approx 3,8$, etapa buffers) y la eficiencia cae en la región oversuscrita $p > 4$, confirmando el techo impuesto por los núcleos físicos. La versión de memoria compartida resulta $30-40\times$ más rápida en cómputo que MPI en un solo nodo. Se derivan además las condiciones de escalabilidad por dos vías: la cota de Brent (sin comunicación, $n \propto \sqrt{p}$) y la isoeficiencia (con comunicación, $n \propto p \log p$). Se *implementa y mide* además la estrategia rival de particionar el train: a igual trabajo, TEST paga $2,5-19\times$ menos overhead de coordinación en un nodo, mientras que TRAIN solo se justifica en clúster cuando X_{train} no cabe en memoria. La precisión (0,9861 en $n = 1797$) es *invariante en p* , validando la equivalencia con el secuencial.

Repositorio: <https://github.com/Snah-s/proyecto-paralela>

Index Terms—KNN, MPI, `mpi4py`, OpenMP, numba, PRAM CREW, *span*, árbol binomial, speedup, isoeficiencia, escalabilidad, Hyper-Threading

I. INTRODUCCIÓN

El algoritmo K-Nearest Neighbors (KNN) es un clasificador supervisado *perezoso* (*lazy learner*) [4]: no construye un modelo en el entrenamiento, sino que *almacena* el conjunto X_{train} y difiere todo el cómputo a la fase de predicción, cuya complejidad es $\mathcal{O}(n_{test} \cdot n_{train} \cdot d)$. Esta estructura hace del KNN un caso ideal de *paralelismo de datos*: la predicción de cada punto de prueba es completamente independiente del resto. Este trabajo (i) formaliza el diseño con un modelo PRAM en notación *pardo* del que se lee el *span*, (ii) implementa y valida tres etapas MPI y una versión de memoria compartida, (iii) mide tiempos de cómputo y comunicación, *speedup*, FLOP/s, precisión y escalabilidad, incrementando p y n , y (iv) usa la complejidad teórica incluida la comunicación $\Theta(\log p)$ para validar los resultados y estimar el número óptimo de

Listing 1. KNN-PRAM (CREW). El *pardo* externo fija el *span*.

```
Entrada: X_tr[n_tr x d], y_tr, X_te[n_te x d], k
Salida : y_hat[n_te]

for 1 <= i <= n_te pardo:           // 1 paso paralelo
    (span externo)
    for 1 <= j <= n_tr:             // O(d) por
        distancia
        D[j] = sum_{f=1..d} (X_te[i,f]-X_tr[j,f])^2
    N_k = k-menores(D)             // O(log n_tr)
    por reduccion
    y_hat[i] = VotoMayoria(y_tr[N_k]) // O(k)
```

procesos. El dataset base (`load_digits`, 1797 imágenes 8×8 , $d = 64$) se escala sintéticamente hasta $n = 20000$ mediante réplicas con ruido gaussiano progresivo, sin fuga de datos (*data leakage*).

II. METODOLOGÍA

II-A. Modelo PRAM CREW y formalismo *pardo*

El diseño sigue el paradigma PRAM CREW (Concurrent Read, Exclusive Write): todos los procesos *leen concurrentemente* X_{train} (solo lectura durante la predicción), y cada proceso *escribe exclusivamente* en su partición disjunta de $\hat{y}$. La lectura concurrente se implementa con `bcast` y la escritura exclusiva se garantiza por la partición disjunta de X_{test} , sin condiciones de carrera ni exclusión mutua.

El Listado 1 expresa el modelo en notación *pardo*, donde cada bucle *pardo* es un *paso paralelo*. Esto permite *leer el span por inspección*: el *pardo* externo sobre los n_{test} puntos es un único paso, su cuerpo (distancia $\mathcal{O}(d)$, selección de k -menores por reducción $\mathcal{O}(\log n_{train})$ y voto $\mathcal{O}(k)$) determina la profundidad.

Trabajo y span. A partir del Listado 1:

$$W(n) = n_{train} n_{test} (3d) = C n^2, \quad C \approx 0,16 \cdot 3d \quad (1)$$

$$T_{\infty} = \mathcal{O}(d + \log n_{train}) = \mathcal{O}(\log n_{train}) \quad (2)$$

donde $d = 64$ es constante y no domina. Por el **Teorema de Brent** [5], con p procesos:

$$\frac{W(n)}{p} \leq T_p \leq \frac{W(n)}{p} + T_{\infty} = \frac{C n^2}{p} + \mathcal{O}(\log n_{train}). \quad (3)$$

II-B. ¿Paralelizar TRAIN o TEST?

Se paraleliza **TEST**. La justificación se apoya en los paradigmas del curso:

1. **KNN es lazy**: no hay fase de entrenamiento con cómputo, *todo* el costo $\mathcal{O}(n_{test} n_{train} d)$ está en la inferencia. No hay nada que paralelizar en “train”.
2. **Independencia**: los puntos de test son mutuamente independientes. Particionar X_{test} produce *escritura exclusiva* (CREW) sin sincronización durante el cómputo.
3. **Particionar TRAIN sería peor**: cada proceso vería solo un subconjunto de vecinos candidatos, obligando a una *reducción global* de los k -menores parciales y a difundir X_{test} a todos, añadiendo comunicación sin reducir el trabajo total.

La estrategia TEST convierte el `gather` en una simple concatenación. La partición de TRAIN solo conviene si n_{train} no cabe en memoria por proceso, lo cual no ocurre en nuestro rango. Esta elección no se asume: se implementa también la estrategia rival y se mide (Sec. III-G).

II-C. Paradigma: MPI (distribuido) vs OMP (compartido)

El patrón (datos solo-lectura compartidos, escritura disjunta) favorece la **memoria compartida**: en OpenMP los hilos comparten X_{train} *sin réplica ni serialización* [6]. MPI, en cambio, paga (i) la *réplica* de X_{train} en cada proceso, (ii) la *serialización* y (iii) la latencia de las colectivas $\Theta(\log p)$. La ventaja de MPI aparece al escalar a *varios nodos*, donde no existe memoria compartida. Por ello se implementan ambos: MPI como vehículo requerido y para el análisis distribuido, y una versión de memoria compartida para la comparación empírica (Sec. III-F). Dado que el GIL de Python impide el paralelismo de hilos real, la contraparte honesta de OpenMP es numba [3]: `@njit(parallel=True)` compila la función a código nativo vía LLVM (sin intérprete ni GIL), y el bucle `prange` sobre los puntos de test se traduce al equivalente de `#pragma omp parallel for` de C/C++, repartiendo las iteraciones (`schedule(static)`) sobre la capa de hilos OpenMP (`libgomp`), `fastmath` habilita la reasociación de punto flotante análoga a `-ffast-math`, y `set_num_threads(p)` equivale a `OMP_NUM_THREADS`. X_{train} se comparte sin réplica (una sola copia en memoria, lectura concurrente CREW real).

Efecto del hardware (4 núcleos / 8 hilos). Los “8 hilos” son SMT/Hyper-Threading: dos hilos lógicos comparten las unidades de ejecución de un núcleo. Como KNN es *compute-bound*, el HT aporta ganancias marginales, por tanto se anticipa y se confirma (Sec. III-A) que el speedup **satura cerca de $p = 4$** , y que $p = 8$ (oversuscrito: 8 procesos sobre 4 núcleos) degrada la eficiencia.

II-D. Etapas de desarrollo y validación

El desarrollo se registra en tres etapas (un solo programa, seleccionable con `STAGE`):

- **loop**: distancia por punto con NumPy, comunicación pickle (`comm.bcast/scatter/gather`).

- **vec**: distancia matricial por *broadcasting* 3D calculada *por lotes* para acotar el pico de memoria (el tensor $n_{local} \times n_{train} \times d$ requeriría ~ 32 GB en $n = 20000$), con `argpartition` $\mathcal{O}(n_{train})$, comunicación pickle.
- **buf**: cómputo idéntico a `vec` + comunicación por buffers (`Bcast/Scatterv/Gatherv`) sin serialización.

Validación: las tres etapas y la versión OMP producen *accuracy idéntica al secuencial* (0,9861 en $n = 1797$) e *invariante en p* , lo que prueba la equivalencia matemática de la paralelización.

II-E. Complejidad, FLOPs y modelo de comunicación

La distancia euclidiana en $\mathbb{R}^d$ requiere d restas, d multiplicaciones y $d - 1$ sumas (+1 raíz, subdominante), es decir

$$\text{FLOP}(d) = 3d = 192, \quad F_{total} = \frac{n_{test}}{p} n_{train} (3d). \quad (4)$$

Comunicación (modelo α - β , árbol binomial). Las colectivas de MPI se implementan con algoritmos de árbol cuya profundidad es $\log p$, con latencia α y costo por elemento β , el broadcast de un mensaje de tamaño m cuesta $\log p (\alpha + \beta m)$. Por tanto

$$T_{comm}(n, p) = \underbrace{\log p (\alpha + \beta n_{train} d)}_{\text{bcast } X_{train}} + \underbrace{\log p \left(\alpha + \beta \frac{n_{test}}{p} d \right)}_{\text{scatter}} + \underbrace{\log p \left(\alpha + \beta \frac{n_{test}}{p} \right)}_{\text{gather}}. \quad (5)$$

El factor $\log p$ (número de rondas del árbol binomial) reemplaza tanto al costo lineal ingenuo $(p - 1)$ como a la suposición de “constante en p ”, es coherente con el DAG del enunciado, $\mathcal{O}(\log p (a + nb))$.

II-F. Generación de datos y entorno

Para escalar n se replica el dataset con ruido gaussiano progresivo $X^{(k)} = X_{orig} + \mathcal{N}(0, \sigma_0 + k \delta \sigma)$, con $\sigma_0 = \delta \sigma = 0,05$ y recorte a $[0, 16]$. El split estratificado 80/20 se hace *antes* de la augmentación (sin *data leakage*). La Tabla I resume el entorno, la CPU es de **4 núcleos físicos / 8 hilos**, coincidiendo con el escenario que plantea el enunciado.

Cuadro I
ENTORNO EXPERIMENTAL

Parámetro	Valor
CPU	Intel i7-1165G7, 4 núcleos / 8 hilos (HT)
RAM	16 GB
OS / MPI	Ubuntu, Open MPI 4.1.6
Python	3.12, mpi4py 3.1.5, numba 0.66
k / iters	3 / 5 por (<i>stage</i> , p , n) (agregado por mediana)
p	{1, 2, 4, 8} ($p > 4$ oversuscrito)

Uso de IA. Se utilizó Claude (Anthropic) [7] como herramienta de apoyo en: (i) exploración de la API MPI de bajo nivel

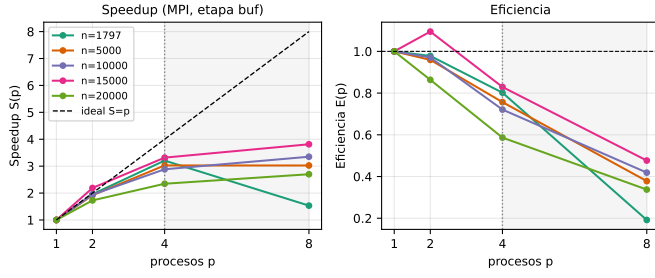


Figura 1. Speedup y eficiencia (etapa buf). Línea punteada: frontera de $p = 4$ núcleos físicos, región sombreada: $p > 4$ oversuscrito (HT).

(Scatterv/Gatherv, counts/displ), (ii) interpretación cualitativa del comportamiento de caché y presión de memoria en la etapa vectorizada, y (iii) discusión exploratoria del modelo de *overhead*. El diseño experimental, la ejecución, las mediciones, el ajuste de modelos y las conclusiones son de autoría propia, la IA no se empleó como fuente de resultados.

III. RESULTADOS Y ANÁLISIS

III-A. Speedup y escalabilidad fuerte

La escalabilidad fuerte mantiene n fijo y varía p . La Tabla II y la Fig. 1 reportan la etapa buf (la de mejor escalado). El speedup crece hasta $p = 4$ y *satura o decae* en $p = 8$: la eficiencia $E(4) \in [0,59, 0,83]$ cae a $E(8) \in [0,19, 0,48]$. Esto es consistente con los 4 núcleos físicos: más allá, el HT y la oversuscripción no aportan cómputo adicional. El mejor resultado es $S(8) = 3,81$ en $n = 15000$.

Cuadro II
SPEEDUP S Y EFICIENCIA E - ETAPA BUF

n	$S(2)$	$E(2)$	$S(4)$	$E(4)$	$S(8)$
1,797	1.96	0.98	3.21	0.80	1.53
5,000	1.92	0.96	3.03	0.76	3.02
10,000	1.94	0.97	2.88	0.72	3.35
15,000	2.19	1.09	3.32	0.83	3.81
20,000	1.73	0.86	2.35	0.59	2.70

III-B. Análisis de comunicación

La Fig. 2 desglosa T_{comm} para $n = 20000$. El bcast de X_{train} crece de 0,04 a 8,5 ms de $p=1$ a $p=4$ y luego se *estabiliza* el comportamiento $\log p$ de la ec. (5), mientras que el gather *explota* a 177 ms en $p = 8$ por la contención de 8 procesos sobre 4 núcleos. Aun así, para $p \leq 4$ la comunicación es $< 0,2\%$ del tiempo total (problema *compute-bound*), la degradación de $p = 8$ es de *recursos*, no de ancho de banda.

III-C. Número óptimo de procesos

Modelando $T(p) = a/p + bp$ (cómputo + overhead) y ajustando a los datos de buf, $n = 20000$ (Fig. 3): $a = 25,2$ s, $b = 0,87$ s, de donde

$$\frac{\partial T}{\partial p} = -\frac{a}{p^2} + b = 0 \Rightarrow p^* = \sqrt{a/b} \approx 5,4. \quad (6)$$

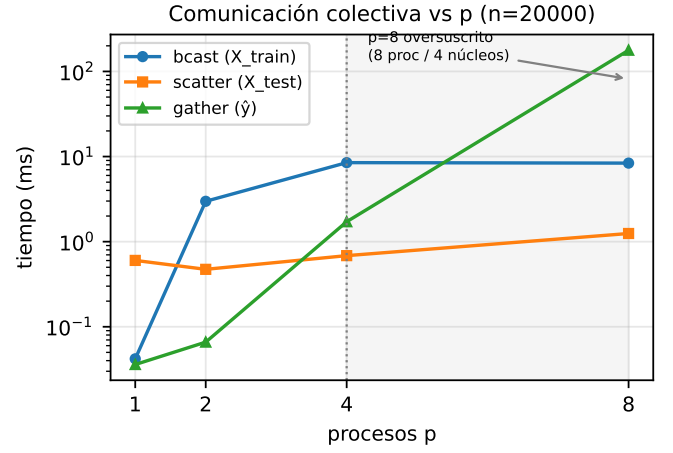


Figura 2. Desglose de la comunicación colectiva vs p ($n = 20000$, escala logarítmica). El factor $\log p$ del árbol binomial y el colapso por oversuscripción en $p = 8$.

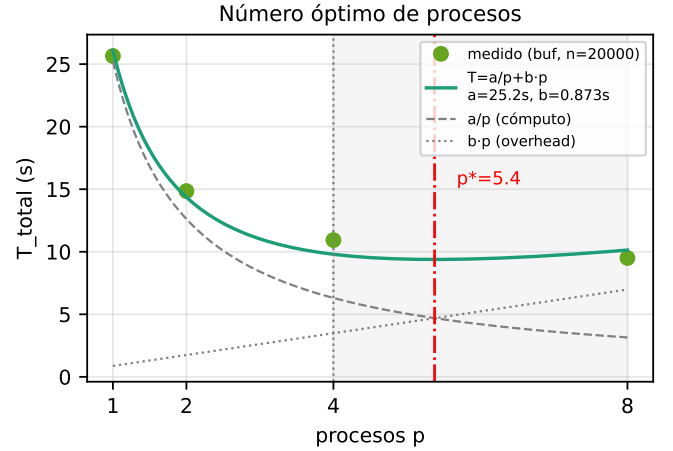


Figura 3. Ajuste $T = a/p + bp$ y número óptimo $p^* \approx 5,4$.

El óptimo cae *entre* los 4 núcleos físicos y los 8 hilos, coherente con la saturación observada: el hardware fija el techo práctico de p (contrasta con el p^* elevado que se obtendría en un clúster con más núcleos).

III-D. Escalabilidad débil: Brent vs isoeficiencia

Se analiza la escalabilidad por dos vías (Fig. 4).

(a) **Cota de Brent (sin comunicación).** Exigir trabajo constante por proceso, $W/p = \text{cte}$, con $W = C n^2$ da $n^2/p = \text{cte}$, esto es

$$n \propto \sqrt{p}. \quad (7)$$

(b) **Isoeficiencia (con comunicación).** Con la función de overhead $T_0 = p T_p - W$ [8] y la comunicación $\Theta(\log p)$ de la ec. (5), el término dominante es $T_0 \approx \beta' p n \log p$. La isoeficiencia $W = k T_0$ da

$$C n^2 = k \beta' p n \log p \Rightarrow n \propto p \log p. \quad (8)$$

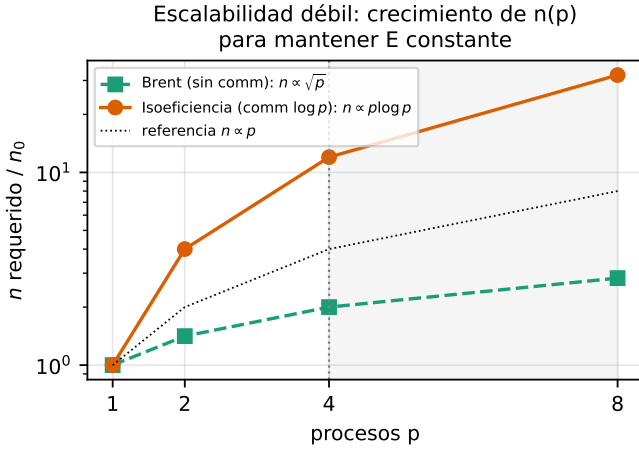


Figura 4. Escalabilidad débil: crecimiento requerido de $n(p)$ para mantener E constante. Brent (sin comm, $\sqrt{p}$) vs isoeficiencia (comm $\log p$, $p \log p$).

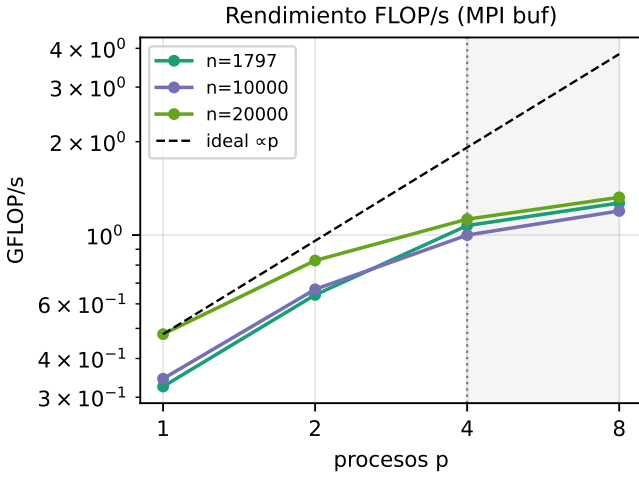


Figura 5. GFLOP/s vs p (etapa buf, escala log-log). Línea punteada: relación ideal $\propto p$.

La comunicación ($\log p$) impone un crecimiento del problema *más rápido* ($p \log p$) que la cota ideal de Brent ($\sqrt{p}$): es el precio de mantener la eficiencia constante cuando se incluye el overhead. La Ley de Amdahl no describe esta escalabilidad, pues es independiente de p , se reserva para el techo de speedup a n fijo.

III-E. Rendimiento en FLOPs

La Fig. 5 muestra GFLOP/s vs p (etapa buf): el rendimiento crece de forma casi lineal hasta $p = 4$ y se aplana en $p = 8$, reflejando la saturación de núcleos. El pico MPI es 1,32 GFLOP/s ($n = 20000$, $p = 8$).

III-F. MPI vs OMP: comparación de paradigmas

La Fig. 6 y la Tabla III comparan la mejor etapa MPI (buf) con la versión de memoria compartida (numba). En un solo

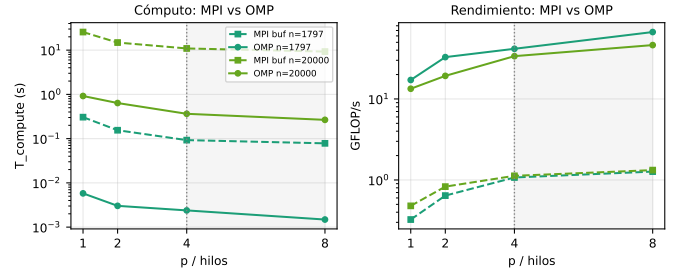


Figura 6. MPI (buf) vs OMP (numba): tiempo de cómputo y GFLOP/s vs p /hilos (escala log).

nodo, la versión OMP es **30-40× más rápida en cómputo**: elimina la réplica de X_{train} , la serialización y la latencia $\log p$, y ejecuta código nativo sin el intérprete de Python. Su rendimiento alcanza 46-67 GFLOP/s (vs $\sim 1,3$ de MPI). Ambos paradigmas saturan de forma similar en ~ 4 núcleos ($S_{OMP}(8) \approx 3,5$), confirmando el techo del hardware. Esto valida empíricamente que, *para un solo nodo con datos solo-lectura*, la memoria compartida es el paradigma más eficiente, MPI se justifica por su escalado a múltiples nodos.

Cuadro III
CÓMPUTO MPI (BUF) VS OMP A $p = 4$

n	T_{MPI} (s)	T_{OMP} (s)	aceleración
1,797	0.093	0.0024	38.8×
5,000	0.726	0.0198	36.7×
10,000	3.072	0.0776	39.6×
15,000	6.119	0.1866	32.8×
20,000	10.921	0.3642	30.0×

III-G. Validación empírica: TEST vs TRAIN

Para *demostrar* (y no solo argumentar) la elección de la Sec. II-B, se implementó la estrategia rival que particiona el *train*: scatter de X_{train} , bcast de X_{test} y una *reducción global* que recolecta los pk candidatos por punto de test y los fusiona en la raíz. Ambas estrategias ejecutan el **mismo trabajo** $n_{test}n_{train}(3d)$ y producen la **misma accuracy**, solo difiere el overhead de coordinación (Tabla IV, Fig. 7, $n = 20000$).

Como el problema es *compute-bound* y el trabajo es idéntico, el tiempo total resulta estadísticamente equivalente (TRAIN es incluso marginalmente más rápido por localidad de caché, $S(8) = 2,78$ vs $2,70$). La ventaja de TEST no está, pues, en la velocidad bruta sino en el **overhead**: su coste de coordinación es 2,5–19,5× menor y, a diferencia del gather de candidatos de TRAIN ($\propto p n_{test} k$), no crece con la carga de consultas. En $p = 8$ el overhead de TRAIN alcanza 1,06s (frente a 0,19s de TEST) y la brecha se ensancha con p , elevando el techo de eficiencia de TEST. TEST es además *altamente paralelizable*, pues su gather es una concatenación, sin fusión global propensa a errores. La partición de TRAIN solo se impone cuando X_{train} **no cabe**

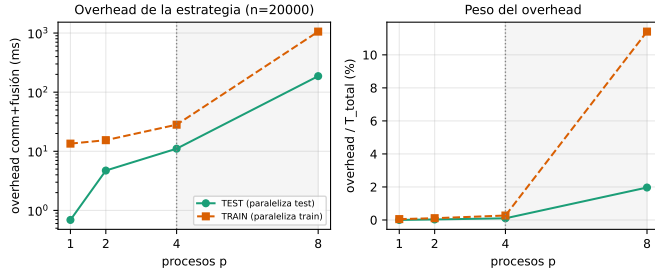


Figura 7. TEST vs TRAIN ($n = 20000$): overhead de coordinación (comunicación + fusión) en valor absoluto (izq.) y como fracción de T_{total} (der.). TRAIN paga 2,5–19× más y crece con p .

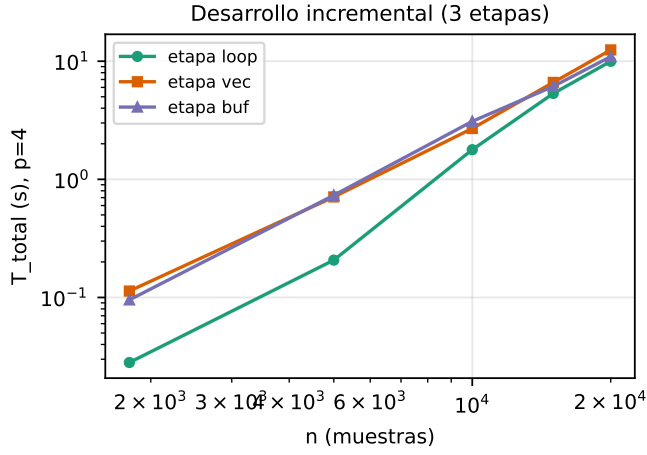


Figura 8. Desarrollo incremental: T_{total} vs n (a $p = 4$) de las tres etapas (escala log-log).

en la memoria de un nodo (régimen de clúster), donde TEST deja de ser viable, cuantificarlo en un clúster real es trabajo futuro.

Cuadro IV
OVERHEAD DE COORDINACIÓN TEST VS TRAIN ($n=20000$, MEDIANA)

p	TEST (ms)	TRAIN (ms)	TRAIN/TEST	S_{TEST}/S_{TRAIN}
1	0.69	13.5	19.5×	1.00 / 1.00
2	4.72	15.4	3.3×	1.73 / 1.80
4	11.1	28.2	2.5×	2.35 / 2.47
8	186.8	1058.1	5.7×	2.70 / 2.78

III-H. Precisión y desarrollo por etapas

La precisión es *invariante en p* para cada n : 0,9861 ($n=1797$), 0,9960 ($n=5000$) y 1,0000 ($n \geq 10000$), crece con n porque las réplicas aumentadas acercan test y train, pero *no depende de p* , validando la equivalencia con el secuencial. La Fig. 8 compara las tres etapas: `loop` es competitiva en absoluto por su bajo overhead, mientras `vec/buf` escalan mejor pero pagan el tráfico de memoria del tensor 3-D.

IV. DISCUSIÓN

Los resultados en hardware de 4 núcleos/8 hilos son *informativos* justo por diferir de un clúster ideal. **Techo por núcleos físicos.** El speedup `buf` sube hasta $S(4) \in [2,35, 3,32]$ y satura o decae en $p = 8$ ($E(4) \approx 0,7$ frente a $E(8) \approx 0,3$): el HT no añade unidades de cómputo y el problema es *compute-bound*, por lo que el $p^* \approx 5,4$ ajustado cae correctamente entre 4 y 8. **Comunicación.** Para $p \leq 4$ es $< 0,2\%$ de T_{total} y sigue el modelo de árbol binomial $\Theta(\log p)$, el salto en $p = 8$ (gather a 177 ms) es contención de recursos por oversubscripción, no ancho de banda.

TEST vs TRAIN. La medición directa (Sec. III-G) matiza y a la vez refuerza la elección de diseño: con trabajo idéntico y problema *compute-bound*, el tiempo total de ambas estrategias es indistinguible en un nodo, pero el overhead de TRAIN es 2,5–19,5× mayor y crece con p (hasta 1,06 s en $p = 8$). La frontera real es de *memoria*: TEST replica X_{train} y solo sirve mientras quepa por nodo, superado ese umbral (clúster), TRAIN se vuelve obligatorio. De forma análoga, la memoria compartida (OMP, 30–40× más rápida en cómputo) domina en un nodo pero se agota al no haber memoria común entre nodos, donde reaparece MPI. Ambos hallazgos convergen en un mismo diseño de clúster: *híbrido* MPI/TRAIN entre nodos, OMP/TEST dentro de cada nodo.

Limitaciones. La CPU es un portátil con *thermal throttling* (mitigado por mediana de 5 repeticiones), el ajuste de p^* usa 4 puntos, insuficientes para distinguir $b p$ de $b \log p$ como modelo de overhead, y el rango $n \leq 20000$ mantiene X_{train} holgadamente en RAM, por lo que la ventaja de TRAIN solo puede *proyectarse*, no medirse, aquí.

V. CONCLUSIONES

Se paralelizó KNN bajo el modelo PRAM CREW, expresado con el formalismo `pardo` para leer el span $T_\infty = O(\log n_{train})$. En hardware de 4 núcleos/8 hilos el speedup MPI satura cerca de $p = 4$ (máximo $S(8) = 3,81$) y la comunicación sigue el modelo de árbol binomial $\Theta(\log p)$, con colapso por oversubscripción en $p = 8$, el óptimo estimado es $p^* \approx 5,4$. La escalabilidad se caracterizó por dos vías complementarias: la cota de Brent ($n \propto \sqrt{p}$, sin comunicación) y la isoeficiencia ($n \propto p \log p$, con comunicación). La versión de memoria compartida resultó 30-40× más rápida que MPI en un solo nodo, evidenciando que para datos solo-lectura y un único nodo OpenMP es el paradigma preferible, mientras MPI se reserva para el escalado multinodo. Al implementar y medir la estrategia rival, se comprobó que particionar TEST paga 2,5–19× menos overhead que particionar TRAIN a igual trabajo, elección que solo se invertiría en clúster cuando X_{train} no cabe por nodo. La precisión, invariante en p , valida la corrección de todas las versiones. Como trabajo futuro: validar en un clúster real tanto el modelo de p^* como el cruce TEST/TRAIN, y solapar comunicación y cómputo con colectivas no bloqueantes.

Algorithm 1 KNN-MPI (etapas loop/vec/buf)

Require: $X_{train}, y_{train}, X_{test}, k, p$ **Ensure:** $\hat{y} \in \mathbb{Z}^{n_{test}}$

```
    ▷ FASE 1 — CARGA (RANK 0)
1: if rank = 0 then
2:   chunks  $\leftarrow$  Split( $X_{test}, p$ )
3: end if

    ▷ FASE 2 — LECTURA CONCURRENTE (CREW)
4:  $X_{train} \leftarrow$  bcast $X_{train}$     ▷ buffer si buf, pickle si no,
    $\Theta(\log p)$ 
5:  $y_{train} \leftarrow$  bcast $y_{train}$ 
6:  $X_{local} \leftarrow$  scatterchunks    ▷  $|X_{local}| = \lceil n_{test}/p \rceil$ 

    ▷ FASE 3 — CÓMPUTO (ESCRITURA EXCLUSIVA)
7: if stage=loop then
8:    $\hat{y}_{local} \leftarrow$  distancia por punto (NumPy)
9: else
10:   $\hat{y}_{local} \leftarrow$  distancia matricial 3-D por lotes +
    argpartition
11: end if

    ▷ FASE 4 — RECOLECCIÓN (RANK 0)
12:  $\hat{y} \leftarrow$  gather $\hat{y}_{local}$ 
13: return  $\hat{y}$  en rank 0
```

REFERENCIAS

- [1] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel *et al.*, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] L. Dalcín, R. Paz, M. Storti, and J. D’Elia, “Mpi for python: Performance improvements and mpi-2 extensions,” *Journal of Parallel and Distributed Computing*, vol. 68, no. 5, pp. 655–662, 2011.
- [3] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A llvm-based python jit compiler,” in *Proc. 2nd Workshop on the LLVM Compiler Infrastructure in HPC (LLVM’15)*, 2015, pp. 1–6.
- [4] T. M. Cover and P. E. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [5] I. Foster, *Designing and Building Parallel Programs*. Addison-Wesley, 1995.
- [6] L. Dagum and R. Menon, “Openmp: An industry standard api for shared-memory programming,” *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, 1998.
- [7] Anthropic, “Claude (modelo de lenguaje),” <https://www.anthropic.com/claude>, 2026, asistencia en exploración de API MPI de bajo nivel y discusión técnica.
- [8] A. Grama, A. Gupta, G. Karypis, and V. Kumar, *Introduction to Parallel Computing*, 2nd ed. Addison-Wesley, 2003, cap. 5: escalabilidad e isoeficiencia.